

Esther Kamal kamel

Unit one

JavaScript

For beginners



**Published in
2026**

The basics of learning JavaScript

what you should know after learning

- understand what JavaScript is and its uses
- Be able to write JS code
- Use variables, data types, and operators
- Write simple functions and control flow statements
- Run JS in a browser console or simple webpage

What is JavaScript?

- JavaScript is a Programming language that makes websites interactive
- it allows you to add dynamic content, respond to user actions, and create applications

Why learn JavaScript?

- One of the most popular programming languages
- Used in almost every website
- Enables you to create websites, apps.
And games

How JavaScript works

1. Runs in the browser or on a server
(Node.js)
2. Can be written directly inside HTML
using **<Script>** tags
3. The browser reads and executes code
line by line

Your first code example

```
1 <Script>
2   console.log("Hello, World!")
3 </Script>
```

- Prints “Hello, World!” in the console

Tips for beginners

- Start with small programs
- Experiment with different values
- Don't be afraid of errors they'll help you learning
- Practice everyday

unit one

(JS Intro & Basics)

- 1.Introduction
- 2.Where to use JS
- 3.Displaying output
- 4.Syntax rules
- 5.Statements
- 6.Comments

(Variables & Operators)

- 1.Variables - Var/Let/Const
- 2.Operators – Arithmetic, assignment, comparison

(Functions & Objects)

- 1.Functions – Defending & calling functions
- 2.Objects – JS objects & Properties
- 3.Events – Handling events

(String & Numbers)

- 1.Strings - JS strings
- 2.Templates – String templates
- 3.Numbers – JS numbers, Math, Random

(Arrays & Dates)

- 1.Arrays JS arrays
- 2.Dates JS date objects

(Logic & 4Loops)

- 1.Booleans - True/False values
- 2.Comparisons – Comparison operators
- 3.If/Else – Conditional statements
- 4.Switch – Switch case
- 5.Loops – For, While, Do-while
- 6.Break – Loop control

(Errors & Modules)

- 1.Errors – Error handling
- 2.Modules – JS modules

**NOTE : we're working with visual code,
you can use any website and right click on
inspect element.**

JavaScript intro & basics

Why we should learn JavaScript?

JavaScript is one of the 3 languages every web developer should know

(HTML, CSS, JavaScript)

1- HTML : Hyper Text Markup Language.

Its the standard language used to create and structure content on the web.

2- CSS : Cascading Style Sheets

It's the language used to style and design web pages that are built with HTML.

**I.e : HTML = the skeleton (Structure),
CSS = the skin, clothes, and colors (style & layout).**

JavaScript = the brain (interactivity & logic)

JavaScript : can change how website behave look and interact with you

JavaScript can change HTML content by : getElementById()

```
<html>
  <body>
    <h2>What can JavaScript do?</h2>
    <p id = "demo">JavaScript can change HTML content</p>
    <button type="button" onclick='document.getElementById("demo").innerHTML = "Hello
    JavaScript!'">Click Me!</button>
  </body>
</html>
```

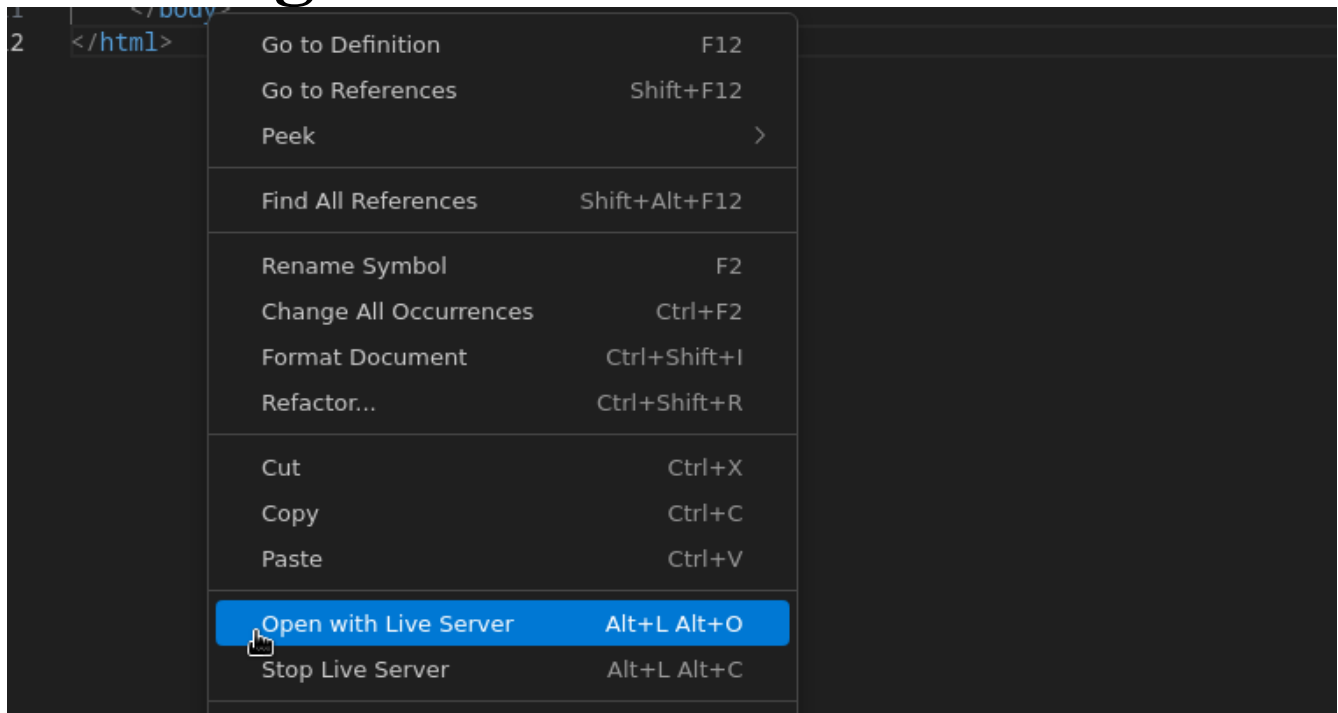
document.getElementById("demo").innerHTML = "Hello JavaScript";

it finds an HTML element on the page with the ID “demo” and then changes its content to display “Hello JavaScript”.

1. **document** = the page
2. **getElementById(“demo”)** = find element with ID “demo”
3. **innerHTML** = the content inside that element
4. **(=)** changes it
5. **“Hello JavaScript”** = the new text string

NOTE : JavaScript accepts both double and single quotes

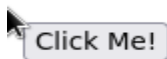
how to get the results :



the results :

What can JavaScript do?

JavaScript can change HTML content



after clicking :

What can JavaScript do?

Hello JavaScript!



The lightbulb test

**first you have to download 2 images
first “offlightbulb” and second
“onlightbulb”**



onlightbulb



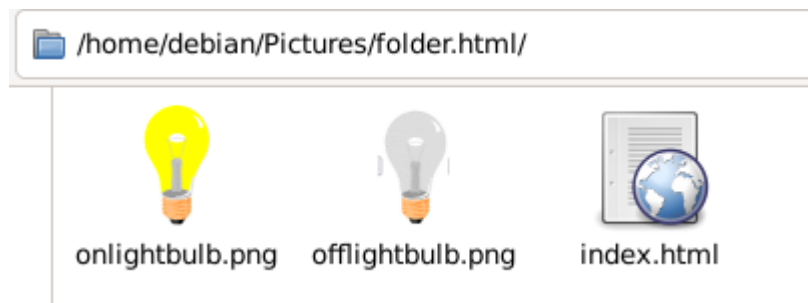
offlightbulb

so lets assume that the image format to be .png

2. second step you have to make a folder

3. add the two images to that folder

4. create an index file and name it “index.html” in
example



it should look like this on linux

NOTE: make sure you've added these images to the index file

Second step (visual code)

on visual code you should write this **code**

```
<> index.html x
<> index.html > html > body > button
1  /home/debian/Pictures/folder.html/
   onlightbulb.png
2  /home/debian/Pictures/folder.html/
3  <!DOCTYPE html>
4  <html>
5  <body>
6
7  <!-- Light bulb image -->
8  
9  <br><br>
10
11 <!-- Buttons to turn on/off -->
12 <button onclick="document.getElementById
   ('myImage').src='./onlightbulb.png'">
13 |   Turn the light on
14 </button>
15 <button onclick="document.getElementById
   ('myImage').src='./offlightbulb.png'">
16 |   Turn the light off
17 </button>|
18
19 </body>
20 </html>
21
```

third step (live server)

What's live server ?

live server : A VS code extension that launches a local web server and auto-refreshes your browser whenever you edit HTML/CSS/JS

how can I download live server on my visual code?

Steps to download live server :

1. open VS Code extensions (Ctrl+Shift+X)
2. search “Live Server” Install (by Ritwick Dey)
3. Open your HTML file – Right click – “Open with Live Server” OR click “Go Live” at the buttom-right
4. Edit files – browser auto-refreshes

Final steps :

1. Open visual studio code
2. Open your index.html file using visual code
3. Add your code.
3. Open the file with live server to see it live

Some code functions :

<!DOCTYPE html> : Declares the document type and version of HTML. (Document type).

<html> : Root element of the HTML document. All other HTML content

<head> : Contains meta-information about the page (not displayed

<meta charset="UTF-8"> : Specifies character encoding for the page.

<title>Light Bulb Example</title> : Sets the title of the page, shown in the browser tab.

<body> : Everything the user sees (text, images, buttons) goes here.

<h2>What Can JavaScript Do?</h2> : Provides a subheading; browsers usually display it larger and bold.

<p style="color:red;">JavaScript can change HTML attribute values.</p> :

function : Paragraph element with inline style

Explanation: Displays text. style="color:red;" makes the text red.

style → inline CSS for this element

color:red; → sets text color to red

** :**

function: it Displays an image.

Explanation:

id="myImage" → uniquely identifies this image for JS manipulation.

src="./offlightbulb.png" → file path of the image (./ = current folder).

alt="Light bulb" → text fallback if image doesn't load.

width="150" → sets image width to 150 pixels.

**

 :**

Function: Line break.

Explanation: Adds vertical spacing between elements. Two
s = two lines of space.

```
<button onclick=
"document.getElementById('myImage').src='./onlightbulb.png'
">Turn the light on</button>:
```

Function: Creates a clickable button that runs JavaScript when clicked.

Explanation:

`document.getElementById('myImage')` → selects the element with `id="myImage"`

`.src='./onlightbulb.png'` → changes the `src` attribute to show a different image

Inner text "Turn the light on" → the label displayed on the button

```
<button onclick=
"document.getElementById('myImage').src='./offlightbulb.png'
">Turn the light off</button>:
```

Function: Same as above, but switches the image back to off.

Explanation: Clicking this button changes the `src` back to `offlightbulb.png`.

Note: Please try these codes by yourself many times to get used to it

How can I change HTML style by JavaScript?

```
document.getElementById("demo").style.fontSize  
= "35px";
```

document Refers to your whole HTML page.

- `.getElementById("demo")` → Finds the element in your page with the ID "demo".
- Example: `<p id="demo">Hello</p>`
- `.style` → Lets you change the CSS style of that element.
- `.fontSize` → Targets the font size of the text inside that element.
- `= "35px";` → Sets the font size to **35 pixels**.

How can I hide HTML elements?

```
document.getElementById("demo").style.display =  
"none";
```

.style.display : Controls how the element is displayed (its CSS display property).

. = "none" ; : Hides the element completely (it disappears from the page layout).

How can I show HTML elements?

```
document.getElementById("demo").style.display  
= "block";
```

display = "block" : makes the element visible again (as a block element, like a `<p>` or `<div>`).

The `<script>` Tag

In HTML, JavaScript code is inserted between `<script>` and `</script>` tags.

First what's the structure of a website?

The structure of the website should look like this :

```
<!DOCTYPE html>
<html>
<head>
|  <title>My First Website</title>
</head>
<body>
|
|  <h1>Hello, World!</h1>
|  <p>This is my first simple website.</p>
|
</body>
</html>
```

so the website should look like this :

Hello, World!

This is my first simple website.

`<!DOCTYPE html>` → tells the browser this is HTML.

- `<html>` → starts the HTML document.
- `<head>` → hidden info (title, styles, scripts).
- `<title>` → the text you see on the browser tab.
- `<body>` → everything the user sees on the page.
- `<h1>` → big heading.
- `<p>` → a paragraph of text.

The arrangement should look like this:

```
<!DOCTYPE html>
<html>
<head>

</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is my first simple website.</p>
</body>
</html>
```

Where do I add the **<script></script>** tags?

A website has 2 main parts:

<head> → info (title, styles, meta)

<body> → what people see

 Where to put **<script>**?

Put it at the end of **<body>** (Most common method)

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Website</title>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is my first simple website.</p>
  <script>
    document.getElementById("demo").innerHTML = "My First JavaScript";
  </script>
</body>
</html>
```

Other way (advanced):

Put in **<head>** but add defer

👉 so it waits until page loads.

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Website</title>
  <script src="script.js" defer></script>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is my first simple website.</p>
</body>
</html>
```

Scripts can be placed in the **<body>**, or in the **<head>** section of an HTML page, or in both.

Small definition:

What is the difference between **<head> & **<body>** ?**

<head> → hidden info, not shown.

Title (tab name), meta, CSS/JS links.

<body> → visible content. Text, images, buttons, links.

How to Use External JavaScript Files?

<script src="myScript.js"></script>

this codes adds an external JavaScript file

lets see how it works:

First step we have to create the folder:



JavaScript tutorial

Second inside the folder, create a file called `index.html`:



index.html

Third Add this code inside the index file:

```
File Edit Search View Document Help
<!DOCTYPE html>
<html>
<head>
  <title>External JS Example</title>
</head>
<body>

<h2>Demo: External JavaScript</h2>

<p id="demo">This is the old text.</p>
<button onclick="myFunction()">Click Me</button>

<!-- Link to external JS file -->
<script src="myScript.js"></script>

</body>
</html>
```

Fourth in the same folder,
create another file called
myScript.js

Make sure the "m" is
small and the "S" is
capital because when
you add it to visual
code it might be case

sensitive

it shouldn't look like this



index.html



MyScript.js

it should look like this

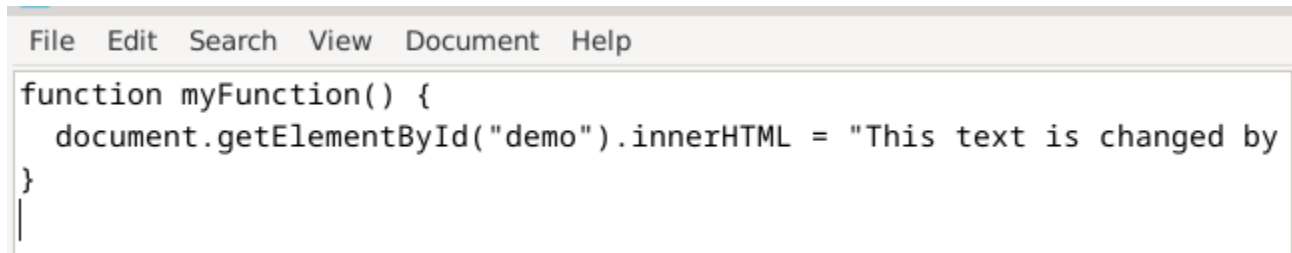


index.html



myScript.js

add this code to the second file:

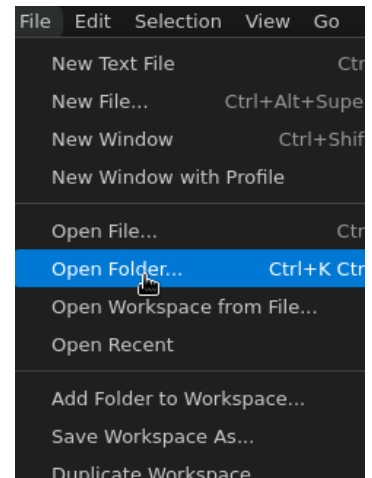


A screenshot of the Visual Studio Code editor interface. The menu bar at the top includes 'File', 'Edit', 'Search', 'View', 'Document', and 'Help'. The editor area contains a JavaScript function:

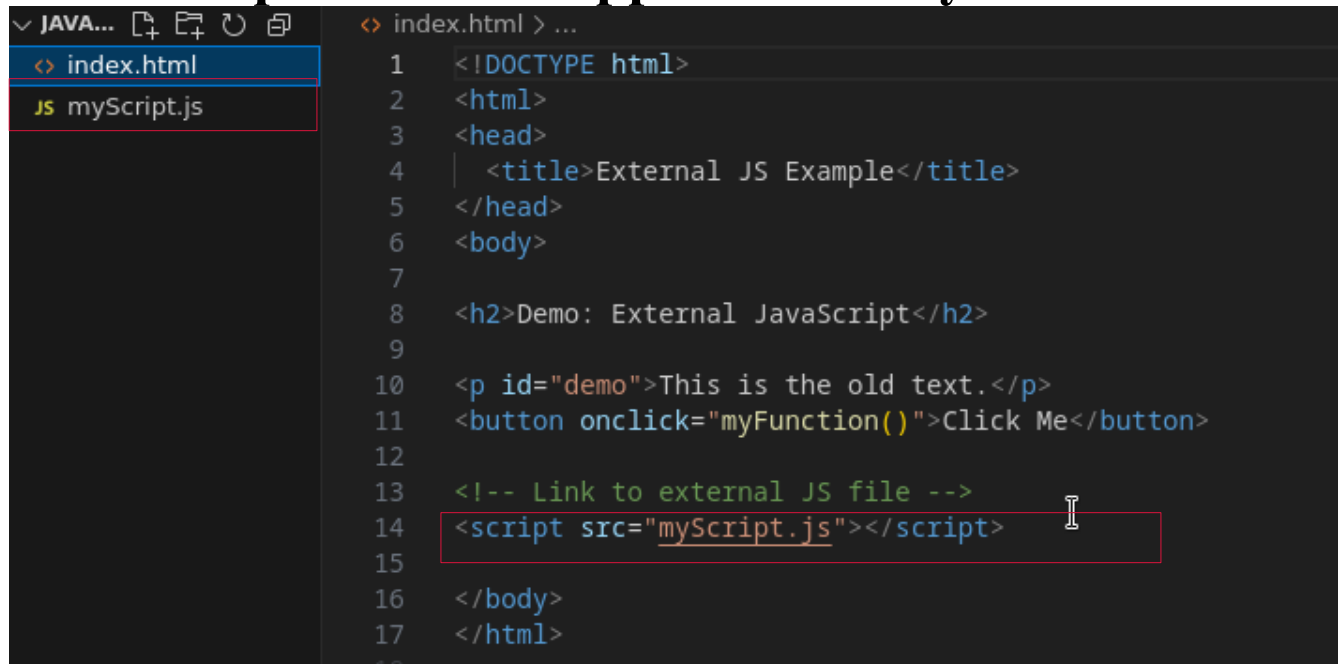
```
function myFunction() {  
  document.getElementById("demo").innerHTML = "This text is changed by  
}
```

Step five open visual code

Open the whole folder (JavaScript tutorial) in VS Code (not just one file).



Let me explain what happened exactly



```
index.html > ...
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>External JS Example</title>
5 </head>
6 <body>
7
8 <h2>Demo: External JavaScript</h2>
9
10 <p id="demo">This is the old text.</p>
11 <button onclick="myFunction()">Click Me</button>
12
13 <!-- Link to external JS file -->
14 <script src="myScript.js"></script>
15
16 </body>
17 </html>
```

As you can see we linked “myScript.js” file with the other index file

now you can easily open with live server and you could see the result

Demo: External JavaScript

This is the old text.



Demo: External JavaScript

This text is changed by external JS!

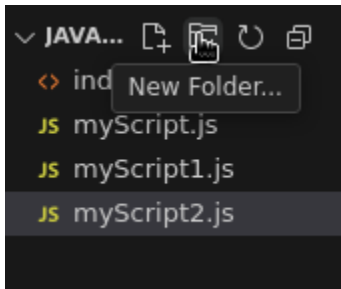


Advantages of External JavaScript

1. **Separation** → HTML = structure, JS = behavior (less mess).
2. **Clarity** → Easier to read and fix (HTML isn't mixed with JS).
3. **Speed** → Browser can save (cache) the JS file → faster loading next time.

You can also use several script tags as you want

And you can add new files and folders as you want using visual code



External References

You can add a website URL to your code like this:

```
<script src="https://www.Esther'sWebsite.com/js/myScript.js"></script>
```

You can add a file path it should look like this:

```
<script src="/js/myScript.js"></script>
```

If the file has no path(beside the html) it should look like this:

```
<script src="myScript.js"></script>
```

JavaScript provides several ways to display data

1. Writing into an HTML element (e.g., `innerHTML`, `innerText`)

`innerHTML` inserts HTML content.

`innerText` inserts plain text (ignores HTML tags).

The code is : `document.getElementById("id")`

2. Writing into the HTML output (using `document.write()`)

Note : If used after the document is loaded, it can overwrite the entire page.

3. Writing into an alert box (`window.alert()`)

Used to display a message in a pop-up dialog box.

4. Writing into the browser console (`console.log()`)

`console.log()` displays the message to the console

its useful for debugging

Using innerHTML

`document.getElementById("id")` Then use the `innerHTML` property to change the HTML content of the HTML element:

Note: Changing the `innerHTML` property of an HTML element is the most common way to display data in HTML.

Using innerText :

`document.getElementById("id").innerText`

What is the difference between `innerText` and `innerHTML`?

- innerHTML

Returns or sets the HTML code inside an element.

It includes tags (like `<b>`, `<i>`, `<p>`, etc.).

If you set `innerHTML`, the browser will interpret it as HTML and render it.

-innerText

Returns or sets only the visible text of an element.

It ignores tags and doesn't render them.

It considers CSS styling (like display:none, hidden text won't show).

Lest see an example:

if we used innerHTML :

```
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "<b>Hello</b> World";
</script>
```

Hello World

if we used innerText :

```
<p id="demo"></p>

<script>
document.getElementById("demo").innerText = "<b>Hello</b> World";
</script>
```

Hello World

Using document.write()

```
<script>
```

```
document.write(5 + 6);
```

```
</script>
```

Note : Using document.write() after an HTML document is loaded, will delete all existing HTML

Lets see an example:

This is a normal page :

```
<!DOCTYPE html>
<html>
<body>

<h1 id="title">My Page</h1>
<p id="text">This is my content.</p>

</body>
</html>
```



My Page

This is my content.

This is the page without using **document.write()**:

```
<!DOCTYPE html>
<html>
<body>

<h1 id="title">My Page</h1>
<p id="text">This is my content.</p>

<button onclick="document.getElementById('text').innerHTML = 'Text
Changed ✓';">
Click Me
</button>

</body>
</html>
```

My Page

This is my content.

Click Me

My Page

Text Changed ✓

Click Me

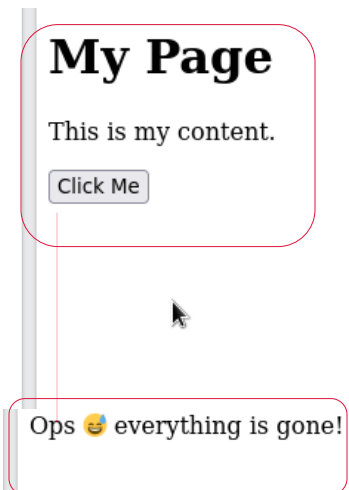
This is while using **document.write()**:

```
<!DOCTYPE html>
<html>
<body>

<h1>My Page</h1>
<p>This is my content.</p>

<button onclick="document.write('Ops 😬 everything is gone!')">Click
Me</button>

</body>
</html>
```



This is after clicking :

Note : The **document.write()** method should only be used for testing.

Using window.alert():

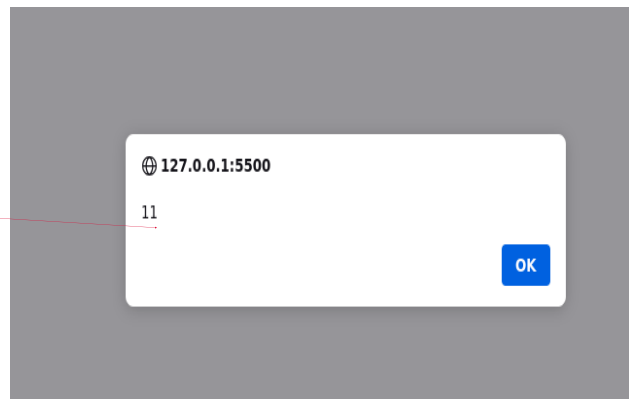
You can use an alert box to display data:

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Web Page</h1>
<p>My first paragraph.</p>

<script>
window.alert(5 + 6);
</script>

</body>
</html> |
```



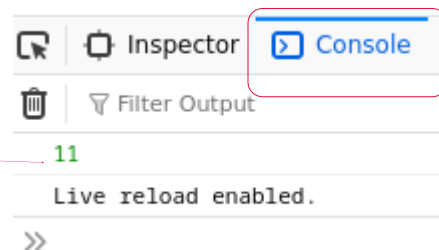
Note: you can skip window word and just use alert()

Using **console.log()**:

```
<!DOCTYPE html>
<html>
<body>

<script>
console.log(5 + 6);
</script>

</body>
</html>
```



Note: For debugging purposes, you can call the **console.log()** method in the browser to display data.

Using JavaScript Print:

The only way to print a web page in JavaScript is with **window.print()**.

Example: **<button onclick="window.print()">Print this page</button>**

you can use CSS print rules to hide or show elements when printing:

```
/* These rules apply only when printing */
@media print {

  /* Hide all buttons when printing */
  button {
    display: none;
  }

  /* Hide all elements with class="no-print" */
  .no-print {
    display: none;
  }

  /* Force all text to be black in print */
  body {
    color: black;
  }
}
```

@media print { ... }

→ means: “use these styles

only when the page is being printed.”

button { display: none; } → hides all buttons in the printed page.

no-print { display: none; } → hides any element with the class no-print.

body { color: black; } → makes all text print in black, even if it's red, blue, etc. on the screen.

Syntax rules

in JavaScript, (var, let, and const) are all used to declare variables (to store data/values).

Lets start with var :

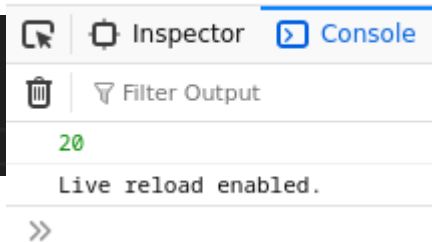
Function-scoped (it belongs to the whole function, not just a block).

Can be **redeclared** and **reassigned**.

Hoisted (moved to the top of its scope automatically).

I.e :

```
<script>var a = 10;
var a = 20; // ✓ allowed (redeclared)
console.log(a); // 20
</script>
```



Second let:

Block-scoped (exists only inside { . . . }).

Can be **reassigned**, but **not redeclared** in the same scope.

Not hoisted in the same way as `var`.

I.e:

```
<script>let b = 10;
b = 20; // ✓ allowed
let b = 30; // ✗ Error (same scope redeclare)
</script>
```

Third const:

Block-scoped.

Cannot be **reassigned** or **redeclared**.

Good for values that never change.

I.e:

```
<script>  
const c = 10;  
c = 20; // ✖ Error  
</script>
```

JavaScript Values

JavaScript has two types of values:

Literals → fixed values

Variables → values that can change

JavaScript Literals

Numbers can be written with or without decimals.

Variables

named values that can change (like `x = 10`). **Strings** are text, written within double or single quotes:

`"Hello"`

`'Hello'`

JavaScript Variables

Variables are **containers** for **storing data values**.

Variables must be **identified** with **unique names**.

`// Define x as a variable`

`let x;`

`// Assign the value 6 to x`

`x = 6;`

JavaScript Identifiers

Identifiers are used to **name variables**, keywords, and functions.

The rules for legal names are the same in most programming languages:

A name must begin with

A name must begin with	A name can contain
------------------------	--------------------

A letter (A-Z or a-z)	A letter (A-Z or a-z)
-----------------------	-----------------------

	A number (0-9)
--	----------------

	)
--	---

A dollar sign (\$)	A dollar sign (\$)
--------------------	--------------------

An underscore (_)	An underscore (_)
-------------------	-------------------

Note : Numbers are not allowed as the first character in names.

This way JavaScript can easily distinguish identifiers from numbers.

JavaScript Keywords

JavaScript enables interaction, and its keywords are reserved words that define actions in the language.

JS Keywords

(let & const) --> make variables

ex:

```
let x = 5;      const name="Java";
```

JS Operators

(=) --> assign value

ex:

```
let sum=x+y;
```

Math operations : (+, -, *, /,)

+ for summation

- for subtractions

* for multiplication

/ for division

JS Expressions

combo of vals, vars, ops → result

ex: $(5+6)*10 \rightarrow 110$

ex: $x*10$

ex: "java"+" "+"script" → "java script"

"" is considered as a space

By the way java script knows if + for names its joining if numbers its addition

JS Comments

// or /* . . . */ -----> ignored by JS

ex:

let x=5; // note

Case Sensitive

lastName $\neq$ lastname

let, not Let or LET

Naming Styles

Hyphen: first-name (not allowed)

Underscore: first_name (allowed)

PascalCase: FirstName (allowed) also called
(**Upper Camel Case**)

camelCase (common): firstName (allowed) also called
(**Lower Camel Case**)

Note: Hyphens are not allowed in JavaScript. They are reserved for subtractions.

JavaScript Statements

A statement = 1 instruction.

Ex:

```
let x, y, z; // declare
```

```
x = 5;      // assign
```

```
y = 6;      // assign
```

```
z = x + y;  // calc
```


S Programs

Program = list of statements.

Run top to bottom (order matters).

In HTML, JavaScript programs are executed by the web browser.

Statement Parts = list of instructions

Built from → values, operators, expressions, keywords, comments.

Ex:

```
document.getElementById("demo").innerHTML = "Java script";
```

Semicolons ;

End statements (optional but recommended).

Can write many on one line:

a=5;

b=6;

c=a+b;

or

a=5; b=6; c=a+b;

Note : JavaScript programs (and JavaScript statements) are often called JavaScript code.

White Space

Extra spaces ignored.

Good style → spaces around operators:

Ex :

```
let x = y + z;
```

```
let x=y+z; ← spaces are ignored here
```

Line Breaks

Break long lines after operators:

```
document.getElementById("demo").innerHTML =  
"Java script!";
```

Note: JavaScript ignores multiple spaces. You can add white space to your script to make it more readable.

Code Blocks {}

Group statements with { }.

Used in functions, loops, if, etc.

Note : The purpose of code blocks is to define statements to be executed together.

Ex: `function myFunction()`

```
{ document.getElementById("demo1").innerHTML = "Hello";  
  document.getElementById("demo2").innerHTML = "How are you?"; }
```

JavaScript Keywords

(Reserved Words)

Reserved → can't be used as variable names.

Special words → define actions.

1. var → old variable

Declares variable (function-scoped, old style).

Example:

```
Var name = "Java";
```

2. let → block variable

Declares variable (block-scoped, modern).

Can change value.

```
let age = 20;
```

```
age = 21; // ok
```

3. const → block constant

Declares constant (block-scoped).

Value cannot change.

```
const PI = 3.14;
```

```
// PI = 3.14159 ❌ error
```

4. if → condition

Runs code if condition is true.

```
if (age >= 18) { console.log("Adult"); }
```

5. switch → cases

Many conditions → choose one.

```
switch(color) {  
  case "red": console.log("Stop"); break;  
  case "green": console.log("Go"); break;  
  default: console.log("Unknown");  
}
```

6. for → loop

Repeats code set times.

```
for (let i=0; i<3; i++) {  
  console.log(i);  
}
```

7. function → declare function

Groups code to reuse later.

```
function greet() {  
  console.log("Hello");  
}  
greet(); // call function
```

8. return → exit function

Sends a value back.

```
function add(a, b) {  
  return a + b;  
}  
console.log(add(2,3)); // 5
```

9. try → handle errors

Catches errors, avoids crash.

```
try {  
  console.log(x); //  error  
} catch (err) {  
  console.log("Error happened!");  
}
```

JavaScript comments

What are comments?

Notes inside code.

Ignored by JS (not executed).

Used to explain, make readable, or disable code during testing.

Types of Comments

1. Single-line (//)

Starts with //.

Everything after // until the end of line = ignored.

Example:

```
<script>  
  // Change heading  
  document.getElementById("myH").innerHTML = "My First Page";  
  
  // Declare x and give it 5  
  let x = 5; // inline comment (after code)  
</script>
```

2. Multi-line (/ * . . . * /)

Starts with / * and ends with * /.

Everything in between = ignored.

Can span multiple lines.

Example:

```
<script>
/*
This block changes:
1. Heading text
2. Paragraph text
*/
document.getElementById("myH").innerHTML = "My First Page";
document.getElementById("myP").innerHTML = "My first paragraph.";
</script>
```

JavaScript Variables

Variables = containers for data

Declared in 4 ways

Modern (preferred): let, const

Old: var, or auto (not recommended).

let vs const vs var

const → value can't change.

let → value can change.

var → old, avoid using.

auto (x=5) → bad practice.

Data Types (8 main)

String → "Hello"

Number → 10, 7.5

Big Integer → large numbers

Boolean → true/false

Undefined → declared, no value

Null → “nothing”

Symbol → unique identifier

Object → key-value pairs

Also: Array ([]), Date (new Date()).

An **array** is a special type of object that stores **lists of values** (in order).

A **Date** object represents a point in time (date + time).

Identifiers (variable names)

Can use letters, digits, _, \$.

Must start with letter / _ / \$.

Case-sensitive (car ≠ Car).

Cannot use JS keywords (let, if, etc.).

Assignment

= means assign (not equal).

Example: $x = x + 5 \rightarrow$ adds 5 to x.

For equality checks: use == or ===.

== (loose equality)

Means: “*Are they similar?*”

Even if they wear different clothes.

Example:

5 (number)

"5" (string / text)

$\rightarrow$ true they're similar

=== (strict equality)

Means: “*Are they exactly the same?*”

Same thing, same clothes.

Example:

5 (number)

"5" (string)

$\rightarrow$ false they're not the same

In declaring

let car; → undefined.

let car = "Volvo"; → with value.

Also you can use many in one line:

let name="John", age=20, car="BMW";

Re-declaring

var → can redeclare.

let / const → cannot redeclare.

So what does redeclare mean?

So in let you can reassign the value but you can't redeclare it

Example:

you can say

let x = 3

x = 4 ← value reassigned

but you cant say

let x = 3

let x = 4 ← value can't be reassigned

we can do that only using var (not recommended).

Var x = 5

var x = 3

Arithmetic

Works like algebra: `let z = x + y;`

Strings + numbers → concatenate:

`"5" + 2 + 3 → "523"`

`2 + 3 + "5" → "55"`

Special Characters

`$` → valid, often used in jQuery.

`_` → valid, often used for private vars.

`#` → valid, used for truly private vars

Golden Rules

Always declare variables.

Use `const` by default.

Use `let` if value will change.

Avoid `var`.



